Spec: Universal Math Graph Token Grammar

Universal Math Graph Token Grammar - Extended Specification

This page documents the **extended universal grammar** for graph-token mathematical representation. It generalizes the balanced-ternary algebra grammar into a **domain-agnostic mathematical IR** capable of expressing algebra, calculus, linear algebra, logic, sets, probability, category theory, and more.
The structural shell remains fixed:

- **Prefix (class):** E, R, O, A
- **Polarity:** +, 0, -
- **Chain marker:** *

Only the vocabulary of entities, relations, and operations expands.

---

1. Extended Entity Space

Entities represent mathematical objects. All follow the same token form:

```
E<polarity>:<entity-id>*
```

Entity categories

```
Variables: X, Y, Z
Numbers: N<int>
Terms: T<int>
Equations: EQ
Functions: F<int>
Matrices: M<int>
Vectors: V<int>
Sets: S<int>
Random variables: RV<int>
Operators: Op<int>
Category objects: Obj<int>
Morphisms: Mor<int>
Custom: <custom-id>
```

---

2. Extended Relation Vocabulary

Relations connect entities. All follow:

```
R<polarity>:<relation-op>*:<entity-ref>:<entity-ref>:<entity-ref>
```

Arithmetic / Algebra

```
R+:mul* R+:add* R-:sub* R-:div* R0:eq*
```

Calculus

```
R+:diff* ; derivative
R+:integrate* ; integral
R0:limit* ; limit relation
R0:eval_at* ; evaluation
```

Linear Algebra

```
R+:matmul* ; matrix multiplication
R+:dot* ; dot product
R+:cross* ; cross product
R0:transpose* ; transpose
R0:inv* ; inverse
R0:det* ; determinant
```

Sets & Logic

```
R+:union* ; set union
R+:intersect* ; set intersection
R-:set_sub* ; set difference
R0:subset* ; subset relation
R0:and* ; logical AND
R0:or* ; logical OR
R0:not* ; logical NOT
R0:implies* ; implication
R0:iff* ; equivalence
```

Probability & Measure

```
R0:prob* ; probability
R0:expect* ; expectation
R0:var* ; variance
R0:dist* ; distribution relation
```

Category Theory

```
R+:compose* ; morphism composition
R0:id* ; identity morphism
R0:assoc* ; associativity
```

---

3. Extended Operation Vocabulary

Operations represent rewrite steps. All follow:

```
O<polarity>:<operation-op>*:<entity-ref>:<entity-ref>
```

Algebra

```
O-:sub* O-:div* O+:add* O0:iso*
```

Calculus

```
O+:apply_diff_rule* ; derivative rule
O+:apply_int_rule* ; integral rule
O0:change_var* ; substitution
O0:limit_simplify* ; limit simplification
```

Linear Algebra

```
O-:row_reduce* ; Gaussian elimination
O0:diag* ; diagonalization
O0:eig_decomp* ; eigen decomposition
```

Logic / Sets

```
O-:simplify_logic* ; logical simplification
O-:simplify_set* ; set simplification
O0:normalize_formula* ; normal form
```

---

4. Universal Grammar Constraints

Polarity discipline

```
+ → structure-building (compose, union, matmul, integrate, diff)
0 → structure-preserving (eq, subset, limit, id, assoc, prob)
- → structure-reducing (sub, div, row_reduce, simplify_logic)
```

Domain consistency

Examples:

- R+:matmul* must connect matrix entities.
- R+:diff* must connect function and variable entities.
- R0:prob* must connect event/set entities and random variables.

These constraints will be enforced in Lean via types.

---

5. Purpose of This Universal Grammar

This grammar is the **candidate universal IR** for representing any mathematical domain in a transformer-friendly, canonical, reversible graph-token format.
It supports:

- algebra
- calculus
- linear algebra
- logic
- set theory
- probability
- category theory
- and any additional domain via new relation/operation symbols

This page serves as the canonical reference for the extended version.